

Coding Agent – Developer Guide

This guide explains how to use the `/implement` agent within the MiDAS GitHub workflow to generate code implementations for MiDAS usecases.

Introduction

The `/implement` agent is a code implementation expert integrated into the MiDAS GitHub workflow. It is triggered through GitHub issue comments and generates complete, high-quality code solutions based on the requirements described in the issue.

The agent analyzes the problem statement, understands repository context, and produces implementation-ready code following best practices such as SOLID principles, proper error handling, input validation, configuration management, and security considerations.

The goal of the `/implement` agent is to accelerate development while maintaining engineering standards and code quality across the organization.

What Are the Use Cases for the Implement Agent

The `/implement` agent should be used when requirements are clearly defined and implementation is ready to begin.

Common Use Cases

- Implementing a new API endpoint
- Adding new business logic to an existing service
- Creating new modules or services
- Adding middleware or integrations
- Implementing authentication or validation logic
- Writing structured code for approved designs



When to Use `/implement` Instead of `/design`

Use `/implement` when:

- The architecture is already finalized.
- The feature scope is clear and approved.
- Acceptance criteria are well defined.
- The change does not require architectural exploration.

Do not use `/implement` when the system design is unclear or when multiple architectural approaches must be evaluated first. In such cases, use `/design`.

How to Use the Implement Agent

The `/implement` agent is triggered from GitHub issue comments.

Step 1: Navigate to the GitHub Issue

Before triggering the agent, the GitHub issue should clearly include:

- Objective
- Functional requirements
- Acceptance criteria
- Constraints (if any)
- Target subsystem or service



Test Issue - Add Health Check Endpoint to SS6 Service #3768

 Open KES1HYD opened this issue 7 minutes ago · 0 comments

 KES1HYD commented 7 minutes ago

Description

We need to add a basic health check endpoint to the SS6 service. This endpoint should allow external systems and monitoring tools to verify that the service is running and responsive.

The endpoint should return a simple success response when the service is healthy.

The goal of this task is to design the endpoint structure, response format, and integration approach within the existing SS6 architecture.

Acceptance Criteria

- A `/health` or `/healthz` endpoint is defined.
- The endpoint returns HTTP 200 when the service is healthy.
- The response format is clearly defined (JSON).
- The design specifies where the endpoint will be implemented (router/controller).
- Any dependency checks (optional DB, cache, etc.) are defined.
- Security considerations are documented (public vs internal access).

Tasks

- Define API contract for health endpoint.
- Identify integration location in SS6 service.
- Define response structure.
- Document assumptions and constraints.

Step 2: Trigger the Agent

In the issue comment section, type:

```
@MiDAS /implement <your query>
```

Example:

```
@MiDAS /implement Implement the health check endpoint as described in this issue.
```



Test Issue - Add Health Check Endpoint to SS6 Service #3768

Open KES1HYD opened this issue 7 minutes ago · 0 comments



KES1HYD commented 7 minutes ago

Description

We need to add a basic health check endpoint to the SS6 service. This endpoint should allow external systems and monitoring tools to verify that the service is running and responsive.

The endpoint should return a simple success response when the service is healthy.

The goal of this task is to design the endpoint structure, response format, and integration approach within the existing SS6 architecture.

Acceptance Criteria

- A `/health` or `/healthz` endpoint is defined.
- The endpoint returns HTTP 200 when the service is healthy.
- The response format is clearly defined (JSON).
- The design specifies where the endpoint will be implemented (router/controller).
- Any dependency checks (optional DB, cache, etc.) are defined.
- Security considerations are documented (public vs internal access).

Tasks

- Define API contract for health endpoint.
- Identify integration location in SS6 service.
- Define response structure.
- Document assumptions and constraints.

Step 3: Review the Generated Implementation

The agent will respond with:

- Code snippets
- File placement suggestions
- Configuration changes (if required)
- Error handling logic
- Input validation (if applicable)
- Security considerations
- Optional test cases (if applicable)

The output is structured and ready to be integrated into the repository after review.



github-actions

bot

commented 18 minutes ago • edited

MiDAS finished @GYH2KOR's task in 4m 23s

[View job](#) • [midas/issue-3768-20260219-0856](#) • [Create PR →](#)

Implementation Complete

Successfully implemented and committed the health check endpoint for SS6 service.

Tasks:

- ✓ Explore SS6 service structure
- ✓ Identify routing patterns
- ✓ Implement health check endpoint
- ✓ Register router in appmain.py
- ✓ Commit changes

Implementation Details

Files Created:

- `ss6-serviceapi-platform/app/api/v1/health/health_router.py` - Health check router with comprehensive documentation
- `ss6-serviceapi-platform/app/api/v1/health/__init__.py` - Module initialization

Files Modified:

- `ss6-serviceapi-platform/app/appmain.py` - Registered health check router (placed first in router list for quick access)

Endpoint Specification:

- **URL:** `GET /services/health`
- **Authentication:** Not required (standard for health checks)
- **Status Code:** `200 OK`
- **Response Format:**

```
{
  "status": "healthy",
  "service": "ss6-serviceapi-platform",
  "version": "1.9.14",
  "build_date": "2026-02-19",
  "timestamp": "2026-02-19T09:00:46.123456Z"
}
```



Features:

- Simple, lightweight health check for monitoring
- Returns service status, version, and build information
- Dynamic timestamp in ISO 8601 format (UTC)
- Compatible with Kubernetes liveness/readiness probes
- Suitable for load balancer health monitoring
- No authentication required (industry standard for health endpoints)
- Comprehensive inline documentation

Design Considerations:

- Follows existing FastAPI router patterns in SS6
- Placed as first router in `appmain.py` for optimal performance
- Loads version from `VERSION.json` for accuracy
- Uses proper HTTP status codes and response types
- Logger integration for debugging
- Detailed docstrings for API documentation

Commit Details:

- **SHA:** `12958a43f107b097d788af80b874c47bf6bb10a9`
- **Branch:** `dev`
- **Message:** "Add health check endpoint to SS6 service"

Testing the Endpoint

Once deployed, you can test the endpoint with:

```
# Using curl
curl http://localhost:5001/services/health

# Using httpie
http GET http://localhost:5001/services/health

# Expected response (200 OK)
{
  "status": "healthy",
  "service": "ss6-serviceapi-platform",
  "version": "1.9.14",
  "build_date": "2026-02-19",
  "timestamp": "2026-02-19T09:00:46.123456Z"
}
```

Use Cases:

- Container orchestration health checks (Kubernetes, Docker)
- Load balancer health monitoring
- Service discovery and registration
- Monitoring and alerting systems (Prometheus, Grafana)
- Quick service availability verification

✅ **Implementation successful!** The health check endpoint is now available at `/services/health` and follows all production-ready best practices.

Resolves [#3768](#)

Ask



What This Validates

Using this example, you can verify that the Implement Agent:

- Understands issue requirements
 - Generates code MiDAS development activity (Usecase and Platform)
 - Follows clean structure and conventions
 - Adds proper error handling and response structure
 - Integrates cleanly within existing architecture
-

Best Practices

To get high-quality implementation outputs:

- Ensure the issue description is precise and unambiguous.
 - Clearly define expected behavior and response formats.
 - Mention the programming language and framework if necessary.
 - Reference any related design document.
 - Specify integration location if known.
-

Code Review Before Merge

Although the `/implement` agent produces structured code, all generated output should:

- Be reviewed by a developer
- Be tested locally
- Pass existing test suites
- Be validated against organizational standards

The agent assists implementation but does not replace peer review or CI validation.



Limitations

- The agent relies on the clarity of the issue description.
 - It does not execute or test code automatically.
 - It assumes repository conventions if not explicitly specified.
 - It does not replace architectural validation.
-

Summary

The `/implement` agent accelerates feature development by generating structured code for MiDAS development activity directly from GitHub issues.

It should be used after design is finalized and requirements are clearly defined, ensuring consistent implementation standards across the organization.

